Bench prototype — Temp + Gravity on an ESP32 dev board

The one build worth doing next. Everything else in `hardware/` specifies custom SMD boards that need a fab run, so nothing in this repo has ever been assembled — and two questions block every other number in the project:

1. **Which accelerometer axis runs along the housing?** Calibration step 1, and currently a guess written into the firmware as an assumption.
2. **Does the float angle sweep usefully across 1.000–1.080?** Depends on ballast and geometry. Nobody has measured it.

Neither is answerable on paper. Both are answerable on a breadboard, for about **$8–17** on top of the board you already own.

(Humidity and CO_2 modules are described in `MULTI_SENSOR_MODULES.md` — deliberately out of scope here.)

Parts

PART	QTY	COST	NOTES
ESP32 dev board	1	—	**D1 Mini ESP32 preferred**; DevKit V1 works but needs an extra regulator, see Power
ADXL345 breakout (GY-291)	1	$2–3	Tilt sensing
DS18B20 waterproof probe	1	$2–3	Right for bench work. The production board uses an on-board MCP9808 instead — the sketch detects which is present, so no code change when you switch
4.7 kΩ resistor	1	~$0.01	DS18B20 1-Wire pull-up
100 kΩ resistors	2	~$0.02	Battery divider
TP4056 module **with protection**	1	$0.50–1	**Count the ICs to tell them apart: protected boards have three** (TP4056 + DW01A + FS8205), unprotected have one. An over-discharged 18650 is a dead 18650
HT7333 or MCP1700 3.3 V LDO + 2× 1 µF	0–1	~$0.30	**DevKit V1 only.** Not needed on a D1 Mini — its own regulator has low enough dropout to run from a cell
18650 holder	1	~$0.50	
18650 cell (~2500 mAh)	1	$2–4	
Breadboard or D1 Mini proto shield	1	$1–3	
Jumper wires	—	$1–2	

Total: ~$9–17. For the gravity sweep you'll also want a hydrometer, table sugar, and a tall container — see Calibrating.

You don't need most of this to start. The axis test needs only the board, the ADXL345 and a USB cable. The DS18B20, battery, TP4056 and divider are for the stages after it.

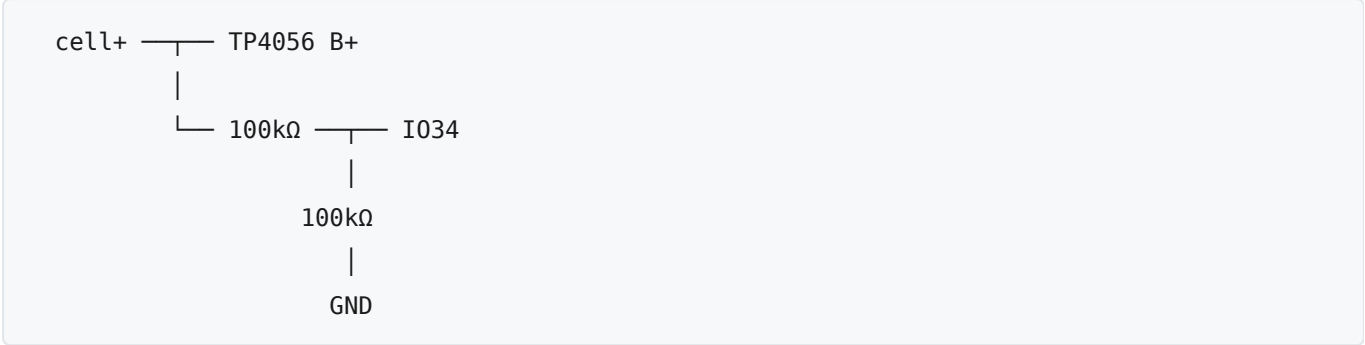
Wiring

Full module-level schematic: `schematic-prototype-d1.html` — every line on it is a wire you physically run.

The firmware's pin map works **unchanged on both boards** — all five pins are broken out on each, with identical silkscreen labels, so `firmware/fermentation_tilt_v1/` flashes and runs as-is.

FIRMWARE	TYPICAL SILKSCREEN	CONNECT TO
`GPIO4`	`IO4` / `D4` / `4`	DS18B20 data (yellow), **plus 4.7 kΩ to 3V3**
`GPIO21`	`SDA` / `IO21`	ADXL345 `SDA`
`GPIO22`	`SCL` / `IO22`	ADXL345 `SCL`
`GPIO34`	`IO34` / `34`	Battery divider midpoint
`GPIO25`	`IO25` / `25`	Divider enable — leave unconnected, see below

Go by GPIO number, not by the label. Silkscreen wording differs between board revisions and clones; the GPIO numbers are what the firmware addresses. Both boards break out all five.



At 4.2 V that puts **2.10 V** on GPIO34 — inside ADC1's range, and it matches the firmware's `* 2.0` exactly. Nothing to change.

DS18B20: red → `3V3`, black → `GND`, yellow → `IO4`. Some probe modules have the 4.7 kΩ already fitted — check before adding a second one.

ADXL345: `VCC` → `3V3`, `GND` → `GND`, `SDA` → `IO21`, `SCL` → `IO22`.

SKIP THE Q1/Q2 DIVIDER GATING

The PCB gates the divider with a PMOS+NPN pair so it draws nothing in sleep. On a breadboard, don't bother: a permanent 200 kΩ divider draws ~21 µA, which is irrelevant beside this board's own sleep current. Wire it straight to GND and leave `IO25` unconnected. The firmware still toggles that pin and waits 30 ms before reading — harmless, and it means **the same sketch runs on both the prototype and the real board**.

POWER — THIS IS WHERE THE TWO BOARDS DIFFER

For the axis test (below), just use USB. No battery, no regulator question, nothing to get wrong. Only the float test needs a battery, because a tethered float can't tilt freely.

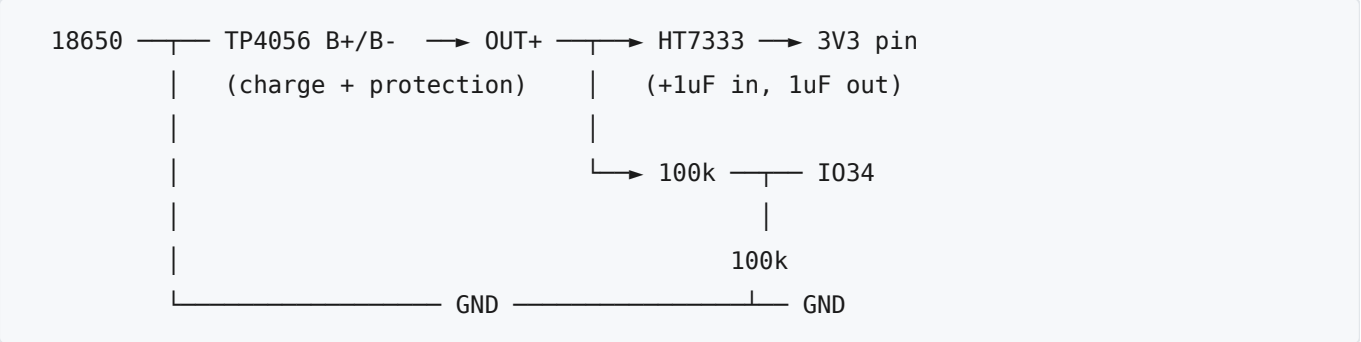
If you have both boards, use the D1 Mini for the float. Its regulator runs directly from a cell, so the power chain is just `cell → TP4056 → OUT+ → 5V pin`, with no extra LDO and nothing to oscillate. The DevKit V1 makes a good second rig for the Hub (temperature only), where it can stay on USB and its sleep current doesn't matter.

D1 Mini ESP32 — cell → TP4056 `B+`/`B-`, TP4056 `OUT+` → the board's `5V` pin. Its ME6211-class LDO drops only ~0.1–0.3 V, so it runs down to a ~3.4–3.6 V cell.

DevKit V1 — a single Li-ion cell will not power it through `VIN`. The board carries an AMS1117, whose dropout is around **1.1 V**: it needs >4.4 V in to hold 3.3 V out, and a Li-ion cell is 4.2 V at full charge and falling. You get brownouts during WiFi transmit, which look like random resets rather than a power fault. Two ways round it:

- **Regulate the cell to 3.3 V yourself** and feed the board's `3V3` pin directly, bypassing the AMS1117. An HT7333 or MCP1700 costs ~$0.30 and drops ~0.1 V with a few µA of quiescent draw.
- **Boost the cell to 5 V** (MT3608-class) into `VIN`. Simpler to buy, wasteful, and noisier.

The full chain on a DevKit V1, with the divider tapping the cell side so it measures battery voltage rather than the regulated rail:



A bare LDO can oscillate without its input and output capacitors — fit both, 1 µF ceramic each is fine.

> **The TP4056 alone is not enough on this board.** Its `OUT+` is the cell voltage, 3.0–4.2 V, so feeding it to `VIN` still browns out. The LDO is the part that makes it work.

> **Never feed the raw cell into `3V3`.** 4.2 V against the ESP32's 3.6 V absolute maximum kills the chip. Feeding `3V3` is only safe from something already regulated to 3.3 V.

> **Never have USB connected at the same time as an external 3V3 feed.** You backfeed the onboard regulator and can damage the regulator, your supply, or both. Unplug USB first.

Charge through the TP4056's USB port, not the board's. Both feed the same rail and will fight. Disconnect the battery before flashing.

---

1. The axis test — five minutes, no housing needed

This is the highest-value thing in the whole project right now, and it needs no float, no sugar, and no waterproofing.

1. Flash `firmware/fermentation_tilt_v1/` with your WiFi and device credentials filled in.
2. Open the serial monitor at 115200.
3. **Hold the board upright**, ADXL345's long axis vertical. The log should print a tilt near **0°**.
4. **Lay it flat.** It should print near **90°**.

If it reads inverted, or sits pinned near 90° in both positions, the board's long axis isn't X. Change which of `x` / `y` / `z` `readTiltDegrees()` treats as the long axis — the code comments mark the exact line.

Do this before collecting a single calibration point. A wrong axis makes every later measurement garbage, which is precisely why the firmware's own calibration procedure puts it first.

---

2. Calibrating the curve

Now you need a float: seal the rig in a candidate housing with ballast, so it floats freely without touching the sides or bottom of the container. A tall narrow vessel — a 1 L measuring cylinder or a tall jar — works better than a wide one.

Making known-gravity solutions. Dissolve table sugar in water. Roughly **0.38 gravity points per gram per litre**, so as starting points:

TARGET SG	SUGAR PER LITRE
1.010	~26 g
1.020	~53 g
1.040	~105 g
1.060	~158 g
1.080	~210 g

Measure each one with a hydrometer rather than trusting the table. You need the hydrometer as the reference anyway, and dissolved volume doesn't track added mass exactly.

Then:

1. Hold every solution at the **same temperature**, and write that temperature down.

2. Record the tilt angle the rig reports in each. Three or four points spanning ~1.000–1.080 is enough.
3. Fit `SG = a·tilt³ + b·tilt² + c·tilt + d`. Zero the unused higher terms for a linear or quadratic fit.
4. Enter the coefficients **and the calibration temperature** on the device page in the dashboard.

> Step 4 matters more than it looks. Gravity is corrected to a 20 °C reference *relative to the calibration temperature*, not to an assumed 20 °C. Calibrating at 24 °C and leaving the field at 20 biases every reading by about half a gravity point, silently. Use hydrometer readings already corrected to 20 °C, too — otherwise a temperature error gets baked into the curve where no later correction can remove it.

What you're really checking is whether the angle moves *enough*. If the rig reads 41° at 1.080 and 39° at 1.010, the concept doesn't work in that housing with that ballast, and no amount of firmware fixes it — you change the ballast or the geometry. Better to learn that now than after ordering 25 housings.

What this rig will not tell you

Battery life. Dev boards keep their USB-serial chip, regulator and power LED powered in deep sleep, so board-level sleep current sits in the **milliamp** range rather than a bare module's ~20 µA. The DevKit V1 is the worse of the two — the AMS1117's quiescent draw alone is ~5 mA, which is why the production design specifies an XC6206 instead. On the same 18650:

SLEEP CURRENT	RUNTIME
~20 µA (bare module — the PCB design)	~5 months
~1.5 mA (D1 Mini ESP32, typical)	~6 weeks
~5 mA (DevKit V1 — AMS1117 quiescent alone)	~2.5 weeks
~10 mA (DevKit V1, with USB-serial chip and LED)	~9 days

Add the LDO headroom loss and prototype runtime could be an order of magnitude short of the product figure. That's a property of the dev board, not the design — don't read it as a failure, and don't quote it as a spec.

Waterproofing or pressure. Both are housing properties, and the housing remains the largest unknown in the project.

If something doesn't work

SYMPTOM	MOST LIKELY CAUSE
`Error: no usable ADXL345 samples`	**I²C address.** The firmware expects `0x53`, which the ADXL345 reports with `SDO` tied **low**. Many GY-291 boards float or pull `SDO` high, giving `0x1D`. Tie `SDO` to GND, or change `ADXL345_ADDR`
Tilt pinned near 90° in every orientation	Wrong axis — see the axis test above
Random resets during WiFi transmit, on a DevKit V1 from battery	AMS1117 dropout — see Power above. It is not a code fault
`Discarding tilt: |a| = …g`	Working as intended: the rig rejects readings whose acceleration isn't 0.8–1.2 g, meaning it was being handled. Put it down and wait a cycle
Erratic or CRC-failing temperatures	Counterfeit DS18B20 — widespread. Suspect the sensor before the code
Device never appears in the dashboard	Check the device ID and secret match the row you created, and that the board is on 2.4 GHz WiFi — the ESP32 has no 5 GHz radio
Gravity column stays empty	Expected until `calibration_c` is set. The view returns `null` rather than a plausible-looking wrong number